Easy Visa Project

Ensemble Techniques

Pauline Zeestraten

Date: February 17, 2024

Contents / Agenda

- Executive Summary
- Business Problem Overview and Solution Approach
- EDA Results
- Data Preprocessing
- Model Performance Summary
- Appendix

Executive Summary

- We observe that 66.8% of visa applicants get certified, and 33.2% denied.
- A large majority of visa applicants have a Bachelor's (40.2%) or Master's degree (37.8%).
- We observe that there is a strong correlation for applicants with a Bachelor's degree with the regions Northeast, West, and South.
- High school, and surprisingly doctorate level education are both negatively correlated, meaning not specifically desirable in any of the regions in the US.
- The vast majority of the applicants does not require job training with 88.4%.
- 90.1% of applicants works on annual salary basis, and only 8.5% works for an hourly wage.

Executive Summary continued

- It is interesting to observe there is a hierarchy among the continents when it comes to visa approval. European applicants are denied the least.
- Asia, the continent where 90% of the applicants hail from, have more applications denied than Europe and Africa.
- Around 55% of the applicants that require job training had no previous work experience. About 45% with prior work experience do require job training.
- We observe that the prevailing wages are higher in the Midwest and Island regions compared to the popular West, Northeast and South regions.
- It seems higher prevailing wages is not enough of a influencing factor in what region applicants end up working.

Business Problem Overview

The Office of Foreign Labor Certification (OFLC) is responsible for administering the US Immigration Programs, and processes job certification applications for employers seeking to bring foreign workers into the United States.

- OFLC grants certifications in those cases where employers can demonstrate that there are not sufficient US workers available to perform the work at wages that meet or exceed the wage paid for the occupation in the area of intended employment.
- The increasing number of applicants every year calls for a Machine Learning based solution that can help in shortlisting the candidates having higher chances of VISA approval.

Business Problem Solution Approach

OFLC has hired our firm EasyVisa for data-driven solutions.

In order to help facilitate the process of visa approvals, and recommend a suitable profile for applicants for whom the visa should be certified or denied based on the drivers that significantly influence the case state, I followed these steps that I will share in this report:

- First, I conducted univariate and bivariate analyses of the data provided by OFLC, to gain a deeper understanding which factors influence the case status of a job seeker.
- Secondly, I used Machine Learning Ensemble Techniques, to develop a classification model that will help OFLC predict the most successful job seeker applicant.

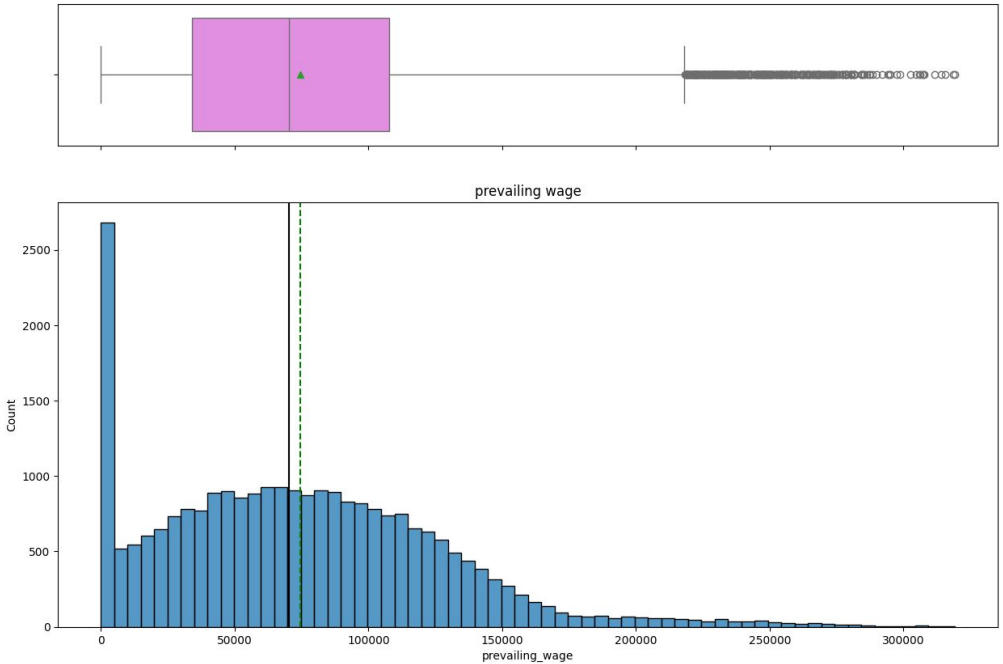
Exploratory Data Analysis - Overview Dataset

- There are 25,480 rows and 12 columns, with no missing or duplicate values.
- The dataset contains 3 numerical and 9 categorical columns.
- We found negative values in the “no_ of_employees” numerical column that we converted to their absolute value (see appendix for details).
- To minimize the size of the dataset to optimize our efficiency in running machine learning models, we dropped the “case_id” column as it contained unique numerical values unrelated to visa status. Secondly, we capped the extreme outliers of the “no_of_employee” column (see appendix and data preprocessing slide for details).

Link to Appendix slide on data background check

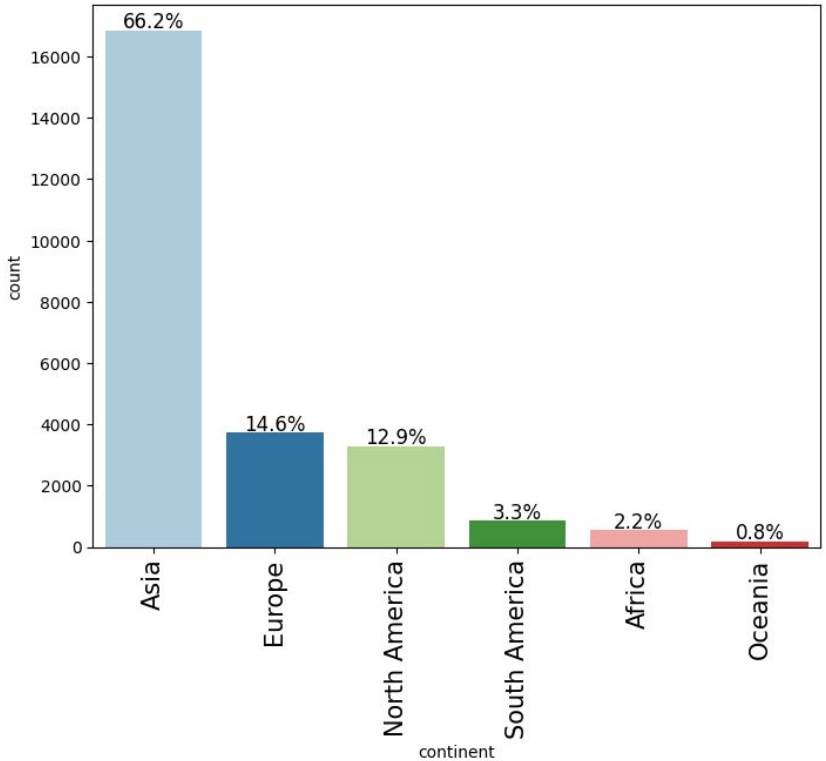
Univariate Analysis - Observations on Prevailing Wages

- We see a heavily right skewed normal distribution of prevailing wages, with the exception of a large peak in count at the lowest wages level.
- When we look at this peak in detail, we find 176 applicants earning less than $100 in prevailing wages
- All 176 applicants work on a hourly basis.



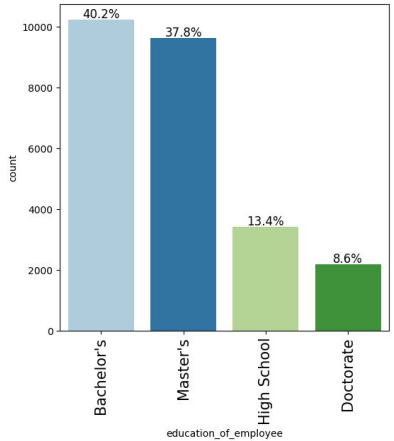
Univariate Analysis - Observations on Continent

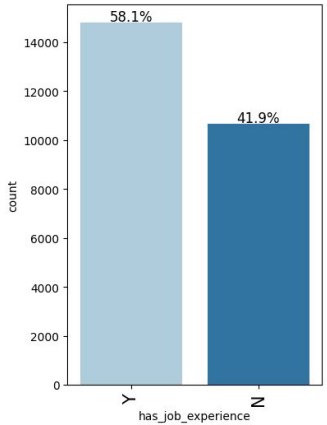
- We see that the majority of job applicants originate from Asia with 66.2%.
- Europe and North America (Canada) trail with 14.6% and 12.9% respectively.



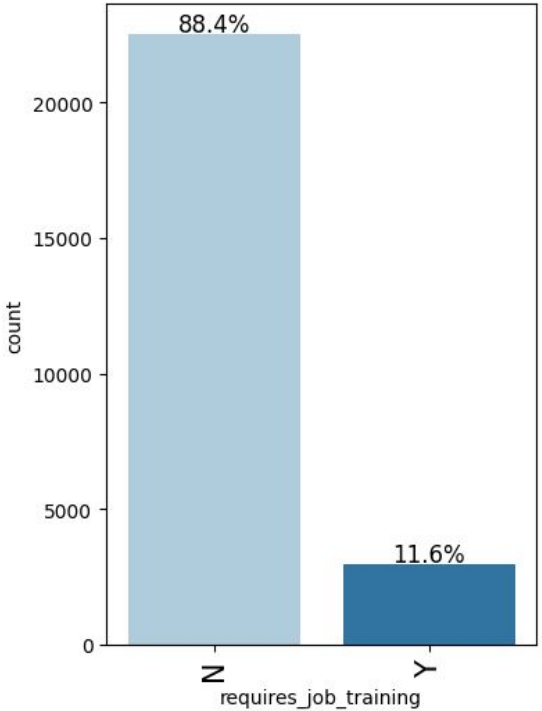
Univariate Analysis - Observations on Education and Job Experience

- We observe that a large majority of visa applicants have a bachelor's (40.2%) or Master's degree (37.8%).
- Only 8.6% has a Doctorate degree.
- 13.4% has only a high school degree.
- We observe less discrepancy in whether applicant employees have job experience or not. 58.1% does have experience, 41.9% does not.

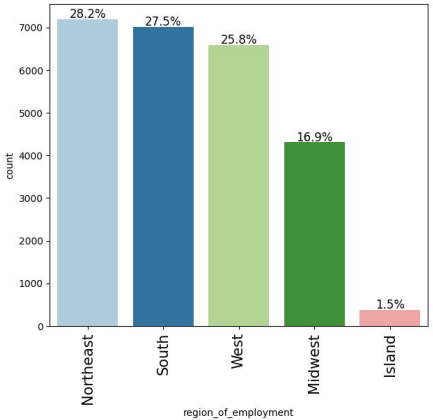




Univariate Analysis - Observations on Job Training + Region of Employment

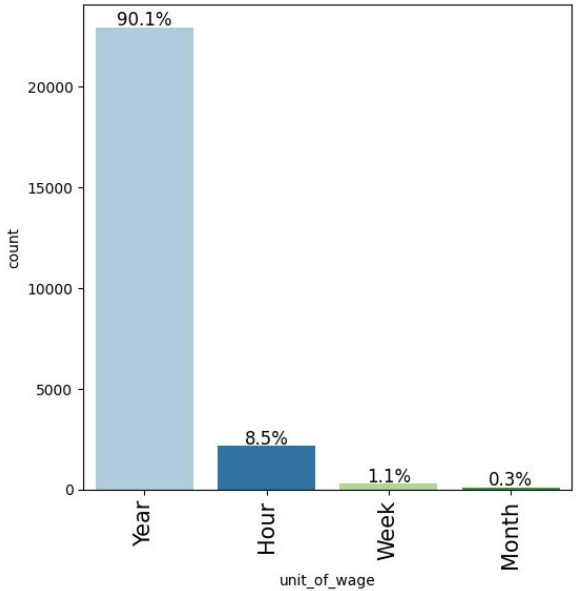


- The vast majority of the applicants does not require job training with 88.4%.

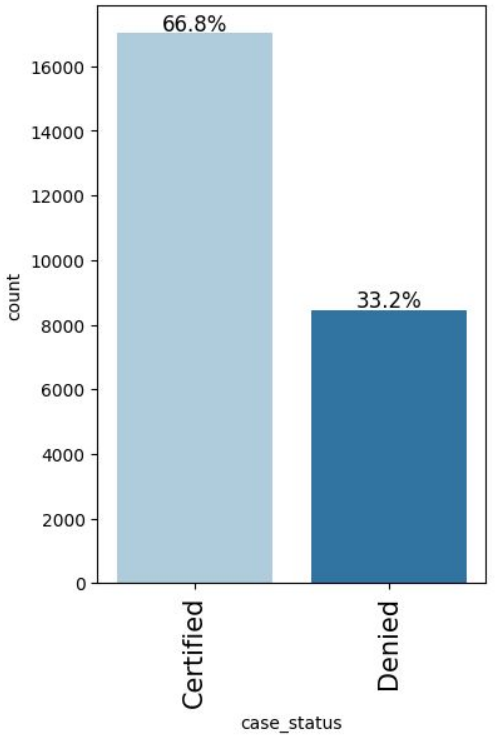


- The distribution of the applicants over the various parts of the US is relatively even.
- A very small minority of applicants work in Hawaii or other US islands, but this is perhaps to be expected given the smaller territories and relative isolation due to nature of island

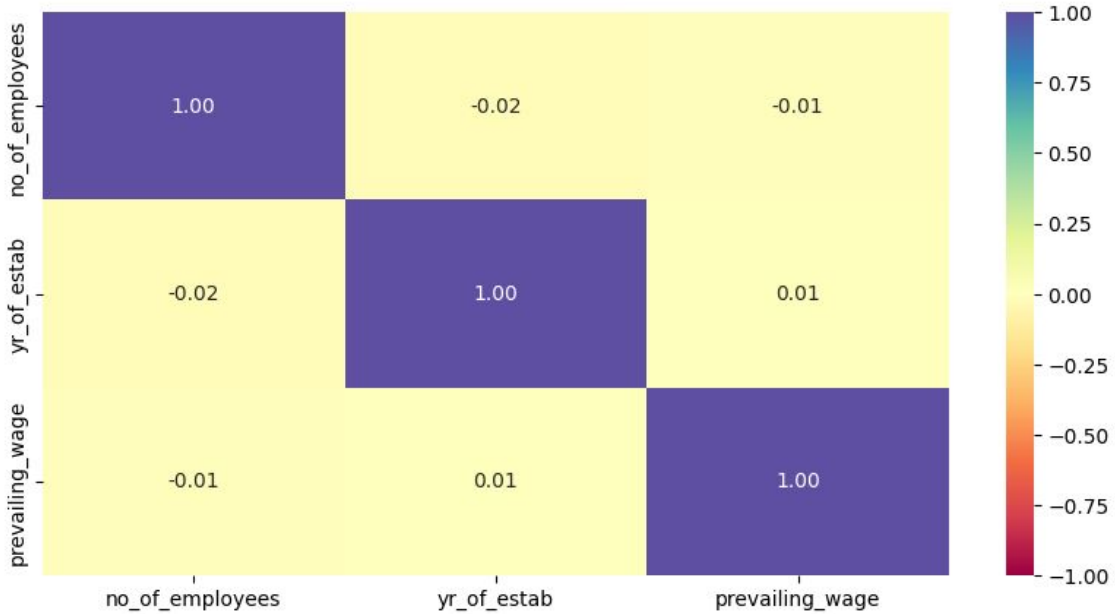
Univariate Analysis - Observations on Unit of Wage and Case Status



- 66.8% of work visa applicants are certified, while 33.2% gets denied.
- 90.1% of applicants works for an annual salary
- Only 8.5% works for an hourly wage



Bivariate Analysis - correlation between numerical variables

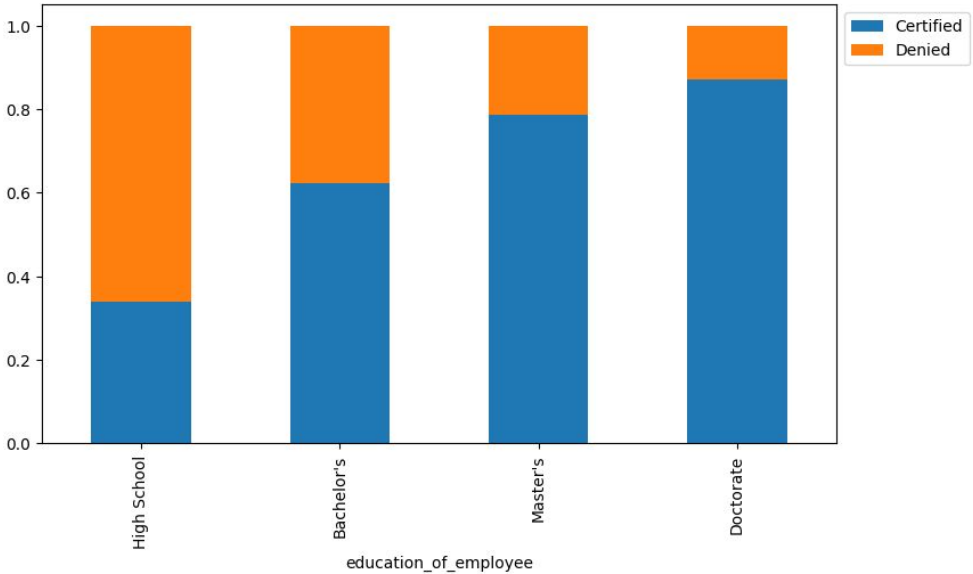


- We find that the correlation between the three numerical features in the dataset is neutral.

  In other words these categories don't correlate or influence each other in a significant way.

Bivariate Analysis - educational level and visa certification

Those with higher education may want to travel abroad for a well-paid job. Let's find out if education has any impact on visa certification



- We observe that level of education is clearly a factor in chances of obtaining a visa. The higher the education level, the higher the chance of obtaining a visa.
- The majority of applicants with only a high school education were denied a visa.

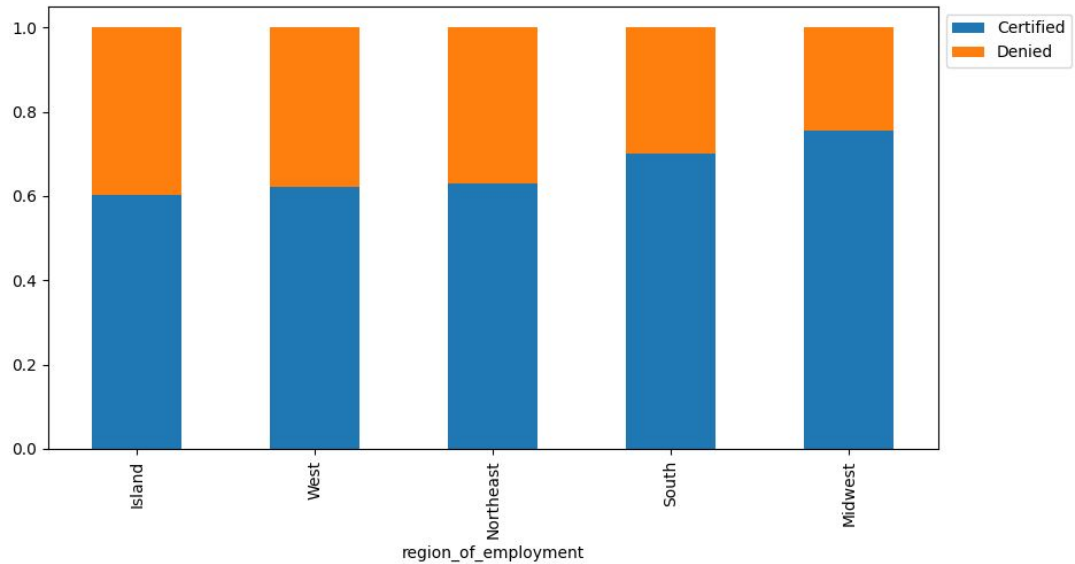
Bivariate Analysis - educational level and regions of employment

Different regions have different requirements of talent having diverse educational backgrounds. Let's analyze it further

- We observe that there is a strong correlation for applicants with a Bachelor's degree with the Northeast, West, and South region in particular.
- Lowest correlation between education degree at all levels is found in the US islands.
- High school, and surprisingly doctorate level education are both negatively correlated, meaning not specifically desired in any of the regions in the US.

Bivariate Analysis - visa certifications and regions of employment

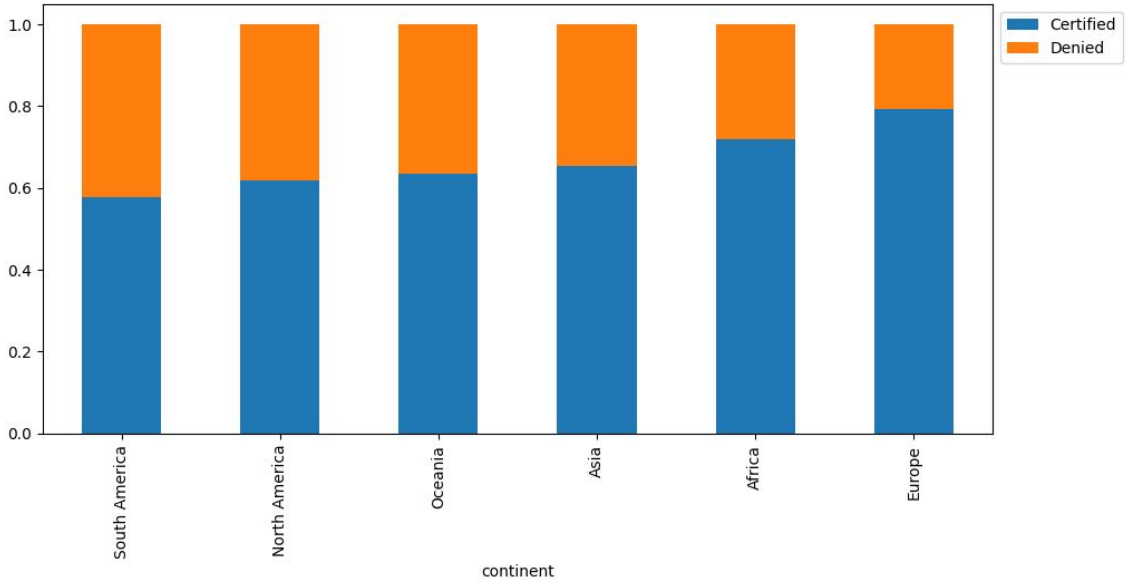
Let's have a look at the percentage of visa certifications across each region



- We observe that visa denial is less than 50% for any region in the US
- However, there is a hierarchy among the regions: The islands, the West, and the Northeast have a similar rate denying applicants, while the South and least of all the Midwest is denying applicants.

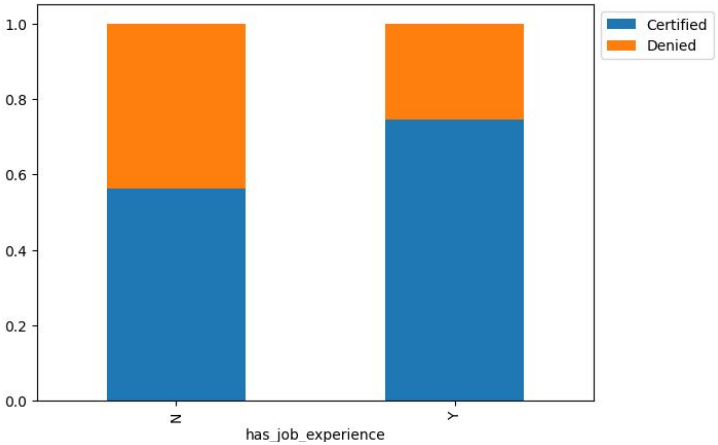
Bivariate Analysis - visa certifications and regions of employment

Lets' similarly check for the continents and find out how the visa status vary across different continents.

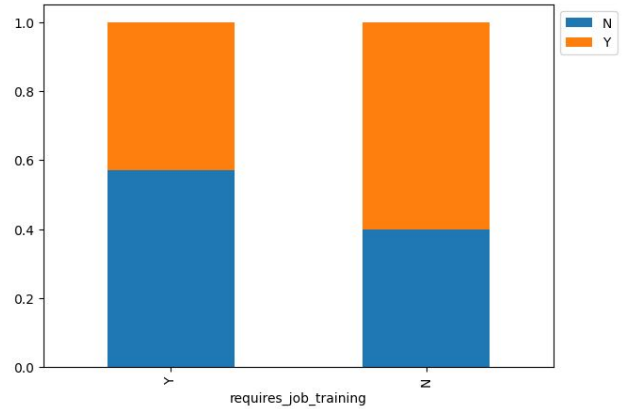


- It is interesting to observe there is a hierarchy among the various continents when it comes to visa approval. European applicants are denied the least.
- Asia, the continent where 90% of the applicants hail from, have more applications denied than Europe and Africa.

Bivariate Analysis - case status, job experience, and job training



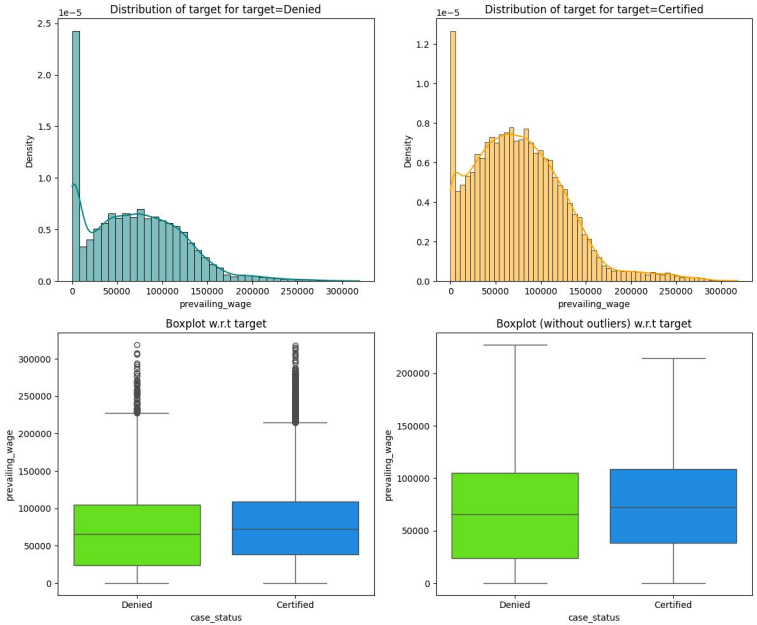
- Experienced professionals might look abroad for opportunities to improve their lifestyles and career development.
- We observe that having prior work experience increases the chance of obtaining a work visa.



- Do employees with prior work experience require any job training?
- We observe that around 55% of the applicants that require job training had no previous work experience. About 45% with prior work experience do require job training.

Bivariate Analysis - case status and prevailing wages

The US government has established a prevailing wage to protect local talent and foreign workers. Let's analyze the data and see if the visa status changes with the prevailing wage

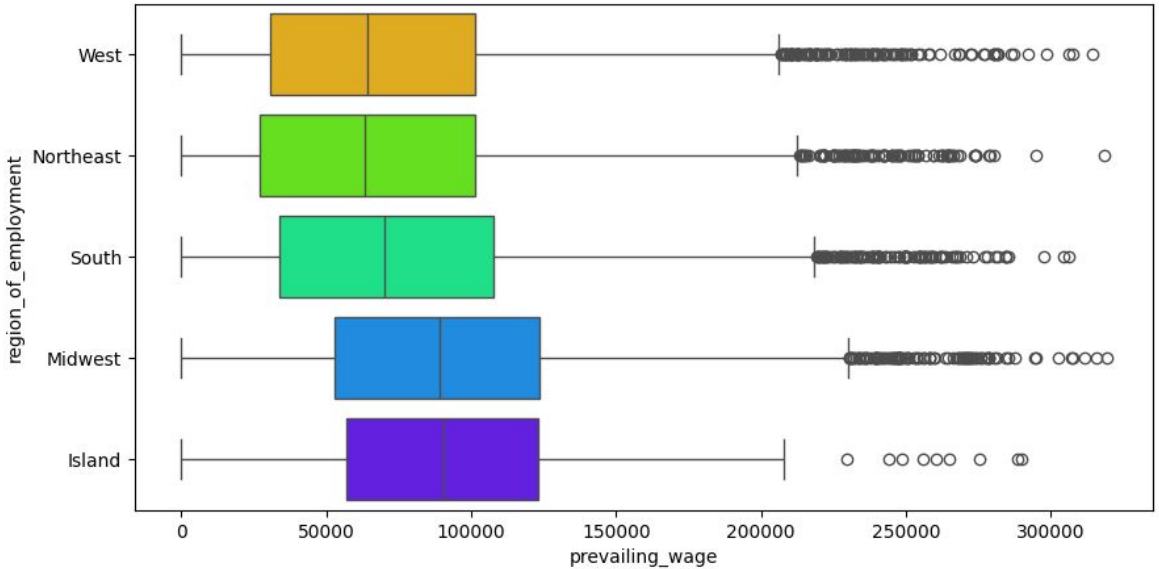


- We observe that for both certified and denied applicants, the distribution is
  - skewed to the right,
  - Equally bell curved / normally distributed
  - except for a big peak in the lowest prevailing wages bracket [<$100] for both categories of applicants.

- We observe the denied applicants have a wider range of prevailing wages.
- The mean salary of certified applicants is slightly higher than that of denied applicants.

Bivariate Analysis - case status and prevailing wages

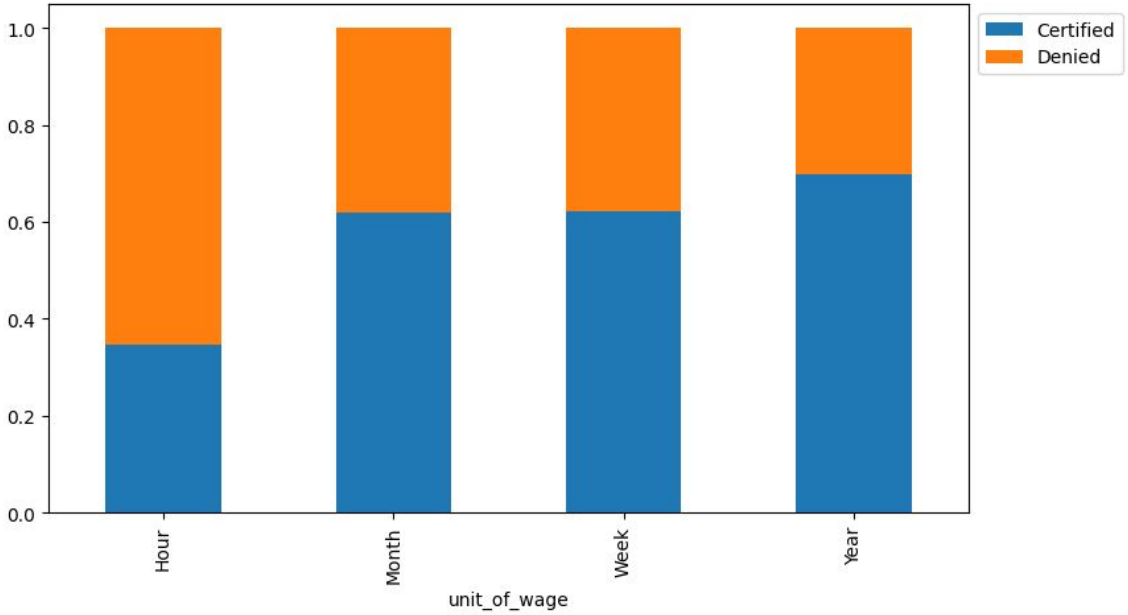
Checking if the prevailing wage is similar across all the regions of the US



- We observe that the prevailing wages are higher in the Midwest and Island regions compared to the popular West, Northeast and South regions.
- It seems higher prevailing wages is not enough of a draw or influencing factor in choosing what region applicants end up working.

Bivariate Analysis - case status and unit of wage

The prevailing wage has different units (Hourly, Weekly, etc). Let's find out if it has any impact on visa applications getting certified.



- We observe that applicants with hourly paid work paid get denied a visa about 65%.
- Monthly and weekly paid applicants get denied almost 40%,
- Annual salaried applicants are denied the least with about 25%

Data Preprocessing

- Duplicate value check

```
# checking for duplicate values
data.duplicated().sum() ## Complete the code to check duplicate entries in the data
```

```
0
```

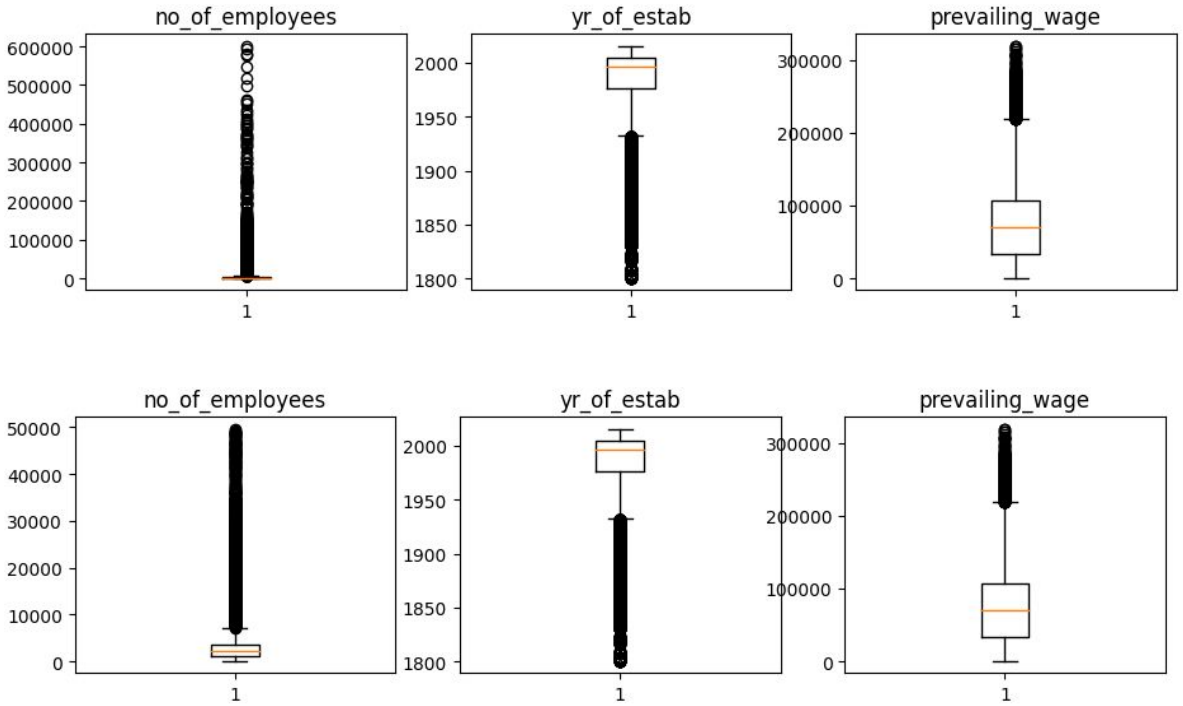
- Missing value treatment - none detected

```
data.isnull().sum()
```

```
continent              0
education_of_employee  0
has_job_experience     0
requires_job_training  0
no_of_employees        0
yr_of_estab            0
region_of_employment   0
prevailing_wage        0
unit_of_wage           0
full_time_position     0
case_status            0
dtype: int64
```

Data Preprocessing continued

- Outlier check (treatment if needed)



The outliers located above the upper whisker of the no_of_employees were treated as follows.

We calculated the value of the upper whisker (7,227) and assigned that value to the outliers >= 50,000

Data Preprocessing continued

Feature engineering

- We found negative values in the “no_ of_employees” numerical column that we converted to their absolute value.
  - ```
    ## Complete the code to check negative values in the employee column
    data.loc[:,'no_of_employees'].sort_values (ascending=True)

    # taking the absolute values for number of employees
    data["no_of_employees"] = abs(data["no_of_employees"]) ## Write the function to convert the values to a positive number
    ```

Data preparation for modeling

- To optimize efficiency in running machine learning models, we dropped the “case_id” column as it contained unique numerical values unrelated to visa status.
  - ```
    data.drop(["case_id"], axis=1, inplace=True) ## Complete the code to drop 'case_id' column from the data
    ```

Data Preprocessing continued

- We want to predict which visa will be certified.
  - ```
    data["case_status"] = data["case_status"].apply(lambda x: 1 if x == "Certified" else 0)
    X = data.drop(['case_status'], axis=1) ## Complete code to drop case status from data
    Y = data["case_status"]
    ```
- Before we proceed to build a model, we'll have to encode categorical features.
  - ```
    X = pd.get_dummies(X, drop_first=True) ## Complete the code to create dummies for X
    ```
- We'll split the data into train and test to be able to evaluate the model that we build on the train data.
  - ```
    # Splitting data in train and test sets
    X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.30,random_state=1, stratify=Y) ## Complete the code to split the data into train and test in the ratio 70:30
    ```
    ```

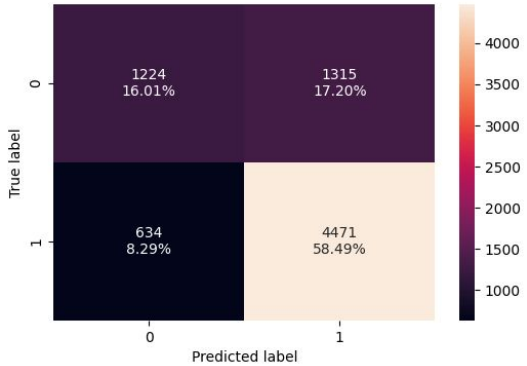
Model Performance Summary

- Overview of final ML model and its parameters
  - `gb_classifier_model_test_perf = model_performance_classification_sklearn(gb_classifier, X_test, y_test) ## Complete the code to check performance for test data`
    `Gb_classifier_model_test_perf`

	Accuracy	Recall	Precision	F1
0	0.745029	0.875808	0.772727	0.821045

The goal is to maximize the F1 score that correlates with lowest possible FPs and FNs.

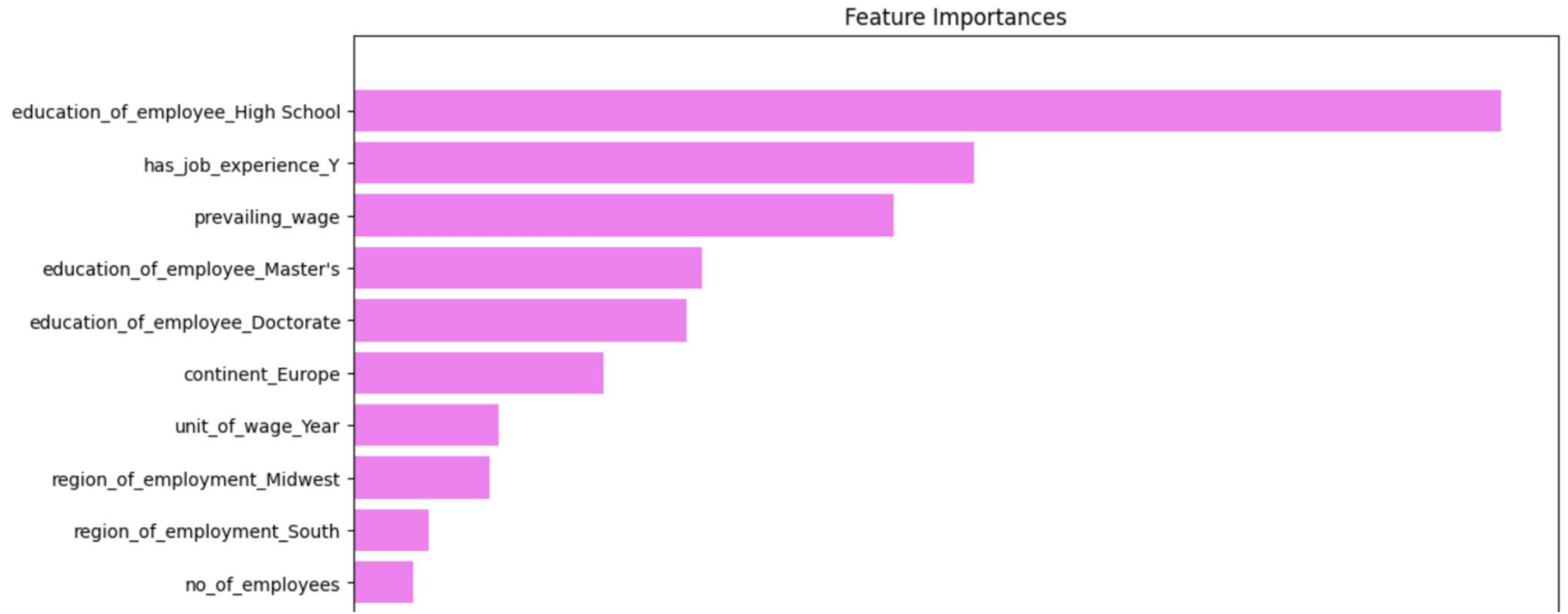
- `confusion_matrix_sklearn(gb_classifier, X_test, y_test) ## Complete the code to create confusion matrix for test data`



The goal is to minimize the false positives and the false negatives.

Feature Importances of Final Model

The following features in hierarchical order play a role of importance in predicting the visa status



Gradient boost classifier model is best performing

Training performance comparison:

	Decision Tree	Tuned Decision Tree	Bagging Classifier	Tuned Bagging Classifier	Random Forest	Tuned Random Forest	Adaboost Classifier	Tuned Adaboost Classifier	Gradient Boost Classifier	Tuned Gradient Boost Classifier	XGBoost Classifier	XGBoost Classifier Tuned	Stacking Classifier
Accuracy	1.0	0.712548	1.0	0.998823	1.0	0.772707	0.738226	0.754878	0.758578	0.754822	0.843575	0.760597	0.766932
Recall	1.0	0.931923	1.0	1.000000	1.0	0.905901	0.887266	0.887518	0.882733	0.886427	0.931503	0.889616	0.883908
Precision	1.0	0.720067	1.0	0.998240	1.0	0.786302	0.760651	0.777141	0.783315	0.777614	0.848979	0.781967	0.791491
F1	1.0	0.812411	1.0	0.999119	1.0	0.841875	0.819094	0.828670	0.830058	0.828463	0.888329	0.832325	0.835151

Testing performance comparison

	Decision Tree	Tuned Decision Tree	Bagging Classifier	Tuned Bagging Classifier	Random Forest	Tuned Random Forest	Adaboost Classifier	Tuned Adaboost Classifier	Gradient Boost Classifier	Tuned Gradient Boost Classifier	XGBoost Classifier	XGBoost Classifier Tuned	Stacking Classifier
Accuracy	0.659210	0.706567	0.659210	0.722920	0.723182	0.741627	0.734171	0.742151	0.745029	0.743197	0.726845	0.744244	0.744113
Recall	0.740255	0.930852	0.740255	0.894417	0.844858	0.886386	0.884427	0.880313	0.875808	0.879726	0.850930	0.878746	0.871303
Precision	0.747133	0.715447	0.747133	0.743043	0.765123	0.764358	0.757932	0.767680	0.772727	0.769007	0.766002	0.770526	0.773969
F1	0.743678	0.809058	0.743678	0.811733	0.803016	0.820862	0.816308	0.820148	0.821045	0.820649	0.806236	0.821085	0.819757

APPENDIX

Data Background and Contents - overview dataset

```
data.shape ## Complete the code to view dimensions of the data
```

(25480, 12)

```
# checking for duplicate values
data.duplicated().sum() ## Complete the code to check duplicate entries in the data
```

0

```
data.describe().T ## Complete the code to print the statistical summary of the data
```

	count	mean	std	min	25%	50%	75%	max
no_of_employees	25480.0	5667.043210	22877.928848	-26.0000	1022.00	2109.00	3504.0000	602069.00
yr_of_estab	25480.0	1979.409929	42.366929	1800.0000	1976.00	1997.00	2005.0000	2016.00
prevailing_wage	25480.0	74455.814592	52815.942327	2.1367	34015.48	70308.21	107735.5125	319210.27

Data Background and Contents - converting negative values

```
## Complete the code to check negative values in the employee column
data.loc[:,'no_of_employees'].sort_values (ascending=True)
```

```
14146       -26
9872        -26
16883       -26
2918        -26
6634        -26
           ...
9587     547172
11317    579004
20345    581468
1345     594472
21339    602069
Name: no_of_employees, Length: 25480, dtype: int64
```

```
# taking the absolute values for number of employees
data["no_of_employees"] = abs(data["no_of_employees"]) ## Write the function to convert the values to a positive number

filter=data['no_of_employees']<0
filter.sum()
```

0

Data Background and Contents - overview categorical variables

```
# Making a list of all
categorical variables
cat_col =
list(data.select_dtypes
("object").columns)
```

```
# Printing number of count of
each unique value in each
column
for column in cat_col:
print(data[column].
value_counts())
print("-" * 50)
```

```
EZYV01       1
EZYV16995    1
EZYV16993    1
EZYV16992    1
EZYV16991    1
            ..
EZYV8492     1
EZYV8491     1
EZYV8490     1
EZYV8489     1
EZYV25480    1
Name: case_id, Length: 25480, dtype: int64
--------------------------------------------------
Asia             16861
Europe            3732
North America     3292
South America      852
Africa             551
Oceania            192
Name: continent, dtype: int64
--------------------------------------------------
Bachelor's     10234
Master's        9634
High School     3420
Doctorate       2192
Name: education_of_employee, dtype: int64
--------------------------------------------------
Y    14802
N    10678
Name: has_job_experience, dtype: int64
--------------------------------------------------
N    22525
Y     2955
Name: requires_job_training, dtype: int64
--------------------------------------------------
Northeast    7195
South        7017
West         6586
Midwest      4307
Island        375
Name: region_of_employment, dtype: int64
--------------------------------------------------
Year     22962
Hour      2157
Week       272
Month       89
Name: unit_of_wage, dtype: int64
--------------------------------------------------
Y    22773
N     2707
Name: full_time_position, dtype: int64
--------------------------------------------------
Certified    17018
Denied        8462
Name: case_status, dtype: int64
--------------------------------------------------
```

Data Background and Contents - dataset overview and univariate analysis

	continent	education_of_employee	has_job_experience
338	Asia	Bachelor's	Y
634	Asia	Master's	N
839	Asia	High School	Y
876	South America	Bachelor's	Y
995	Asia	Master's	N
...	...	...	...
25023	Asia	Bachelor's	N
25258	Asia	Bachelor's	Y
25308	North America	Master's	N
25329	Africa	Bachelor's	N
25461	Asia	Master's	Y

176 rows × 11 columns

```
# checking the number of unique values
data["case_id"].nunique() ## Complete the code to check unique values in the mentioned column

data.drop(["case_id"], axis=1, inplace=True) ## Complete the code to drop 'case_id' column from the data

histogram_boxplot(data, 'prevailing_wage') ## Complete the code to create histogram_boxplot for prevailing wage

# checking the observations which have less than 100 prevailing wage
data.loc[data['prevailing_wage']<100] ## Complete the code to find the rows with less than 100 prevailing wage

## We see 176 rows have job applicants earning less than 100 prevailing wage.
```

Data Background and Contents - univariate analysis

```
data.loc[data["prevailing_wage"] < 100, "unit_of_wage"].nunique() ## Complete the code to get the count of the values in the mentioned column
```

1

```
data.loc[data["prevailing_wage"] < 100, "unit_of_wage"].unique()
```

array(['Hour'], dtype=object)

```
labeled_barplot(data, "continent", perc=True)
```

```
labeled_barplot(data,'education_of_employee', perc=True) ## Complete the code to create labeled_barplot for education of employee
```

```
labeled_barplot(data, 'has_job_experience', perc=True)## Complete the code to create labeled_barplot for job experience
```

```
labeled_barplot(data,'requires_job_training', perc=True) ## Complete the code to create labeled_barplot for job training
```

Data Background and Contents - univariate analysis

```
labeled_barplot(data,'region_of_employment', perc=True) ## Complete the code to create labeled_barplot for region of employment

labeled_barplot(data,'unit_of_wage', perc=True) ## Complete the code to create labeled_barplot for unit of wage

labeled_barplot(data, 'case_status', perc=True) ## Complete the code to create labeled_barplot for case status
```

- bivariate analysis

```
cols_list = data.select_dtypes(include=np.number).columns.tolist()
plt.figure(figsize=(10, 5))
sns.heatmap(
data[cols_list].corr(), annot=True, vmin=-1, vmax=1, fmt=".2f", cmap="Spectral"
) ## Complete the code to find the correlation between the variables
plt.show()
```

Bivariate Analysis - educational level, case status vs regions of employment continent vs case status

```
plt.figure(figsize=(10, 5))
sns.heatmap(pd.crosstab(data['education_of_employee'],data['region_of_employment']),
annot=True,
fmt="g",
cmap="viridis"
) ## Complete the code to plot heatmap for the crosstab between education and region of employment

plt.ylabel("Education")
plt.xlabel("Region")
plt.show()

stacked_barplot(data,'region_of_employment','case_status') ## Complete the code to plot stacked barplot for region of employment and case status

stacked_barplot(data,'continent', 'case_status') ## Complete the code to plot stacked barplot for continent and case status
```

Bivariate Analysis - case status vs job experience

```
stacked_barplot(data,'has_job_experience','case_status') ## Complete the code to plot stacked barplot for job experience and case status
```

```
stacked_barplot(data,'requires_job_training','has_job_experience') ## Complete the code to plot stacked barplot for job experience and requires_job_training
```

```
distribution_plot_wrt_target(data,'prevailing_wage','case_status') ## Complete the code to find distribution of prevailing wage and case status
```

```
plt.figure(figsize=(10, 5))
sns.boxplot(data, x='prevailing_wage', y='region_of_employment', palette='gist_rainbow') ## Complete the code to create boxplot for region of employment and prevailing wage
plt.show()
```

```
stacked_barplot(data,'unit_of_wage', 'case_status') ## Complete the code to plot stacked barplot for unit of wage and case status
```

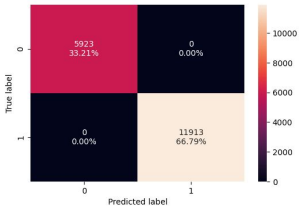
Model Building - Bagging - Decision Tree

```
model = DecisionTreeClassifier(random_state=1)
## Complete the code to define decision tree classifier with random state = 1
model.fit(X_train,y_train)
## Complete the code to fit decision tree classifier on the train data

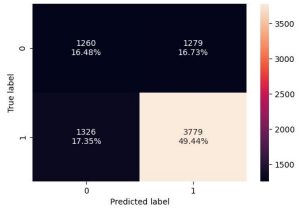
confusion_matrix_sklearn(model, X_train, y_train)
## Complete the code to create confusion matrix for train data
decision_tree_perf_train = model_performance_classification_sklearn
(model, X_train, y_train) ## Complete the code to check performance on train data
decision_tree_perf_train
confusion_matrix_sklearn(model, X_test, y_test)
## Complete the code to create confusion matrix for test data
decision_tree_perf_test = model_performance_classification_sklearn
(model, X_test, y_test) ## Complete the code to check performance for test data
decision_tree_perf_test
```

Observation: Training data of Decision Tree is overfitted. Model is unreliable.

```
DecisionTreeClassifier(random_state=1)
```



	Accuracy	Recall	Precision	F1
0	1.0	1.0	1.0	1.0

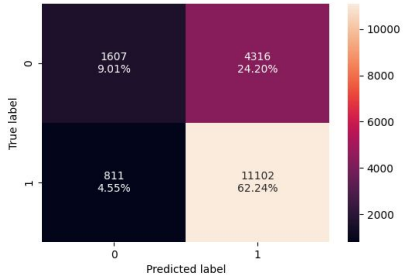


	Accuracy	Recall	Precision	F1
0	0.65921	0.740255	0.747133	0.743678

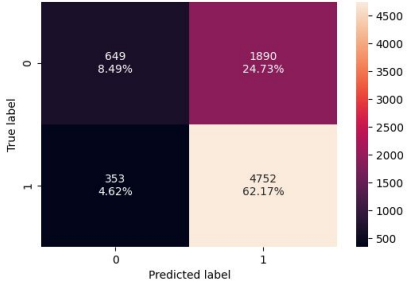
Model Building - Bagging - Decision Tree tuned

```
# Run the grid search
grid_obj = GridSearchCV(dtree_estimator, parameters, scoring=scorer, n_jobs=-1)
## Complete the code to run grid search with n_jobs = -1
grid_obj = grid_obj.fit(X_train, y_train)
## Complete the code to fit the grid_obj on the train data
confusion_matrix_sklearn(dtree_estimator, X_train, y_train)
## Complete the code to create confusion matrix for train data on tuned estimator
dtree_estimator_model_train_perf = model_performance_classification_sklearn
(dtree_estimator, X_train, y_train)
## Complete the code to check performance for train data on tuned estimator
Dtree_estimator_model_train_perf
confusion_matrix_sklearn(dtree_estimator, X_test, y_test)
## Complete the code to create confusion matrix for test data on tuned estimator
dtree_estimator_model_test_perf = model_performance_classification_sklearn
(dtree_estimator, X_test, y_test)
## Complete the code to check performance for test data on tuned estimator
dtree_estimator_model_test_perf
```

```
DecisionTreeClassifier
DecisionTreeClassifier(class_weight='balanced', max_depth=10, max_leaf_nodes=2,
                       min_impurity_decrease=0.0001, min_samples_leaf=3,
                       random_state=1)
```



	Accuracy	Recall	Precision	F1
0	0.712548	0.931923	0.720067	0.812411



	Accuracy	Recall	Precision	F1
0	0.706567	0.930852	0.715447	0.809058

Observation: Much improved performance

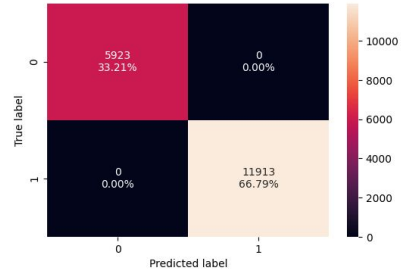
Model Building - Bagging -Bagging Classifier

```
bagging_classifier = (DecisionTreeClassifier (random_state=1))
## Complete the code to define bagging classifier with random state = 1
bagging_classifier.fit(X_train, y_train)
## Complete the code to fit bagging classifier on the train data

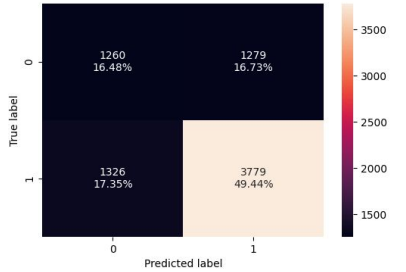
confusion_matrix_sklearn(bagging_classifier, X_train, y_train)
## Complete the code to create confusion matrix for train data
bagging_classifier_model_train_perf =
model_performance_classification_sklearn(bagging_classifier, X_train, y_train)
## Complete the code to check performance on train data
Bagging_classifier_model_train_perf
confusion_matrix_sklearn(bagging_classifier, X_test, y_test)
## Complete the code to create confusion matrix for test data
bagging_classifier_model_test_perf = model_performance_classification_sklearn
(bagging_classifier, X_test, y_test)
## Complete the code to check performance for test data
bagging_classifier_model_test_perf
```

Observation: Model is overfitted, and unreliable.

```
DecisionTreeClassifier
DecisionTreeClassifier(random_state=1)
```



	Accuracy	Recall	Precision	F1
0	1.0	1.0	1.0	1.0

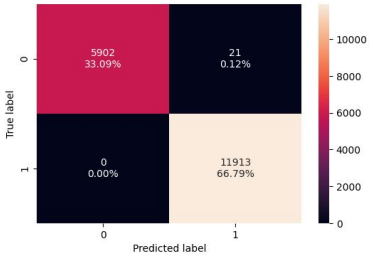


	Accuracy	Recall	Precision	F1
0	0.65921	0.740255	0.747133	0.743678

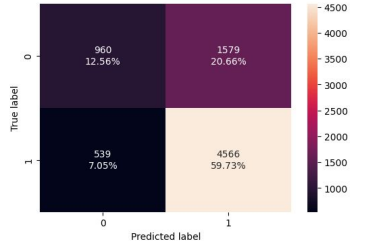
Model Building - Bagging -Bagging Classifier Tuned

```
# Run the grid search
grid_obj = GridSearchCV(bagging_estimator_tuned, parameters, scoring=acc_scorer, cv=5)
## Complete the code to run grid search with cv = 5
grid_obj = grid_obj.fit(X_train, y_train)
## Complete the code to fit the grid_obj on train data
confusion_matrix_sklearn(bagging_estimator_tuned, X_train, y_train)
## Complete the code to create confusion matrix for train data on tuned estimator
bagging_estimator_tuned_model_train_perf =
model_performance_classification_sklearn(bagging_estimator_tuned, X_train, y_train)
## Complete the code to check performance for train data on tuned estimator
bagging_estimator_tuned_model_train_perf
confusion_matrix_sklearn(bagging_estimator_tuned, X_test, y_test)
## Complete the code to create confusion matrix for test data on tuned estimator
bagging_estimator_tuned_model_test_perf =
model_performance_classification_sklearn(bagging_estimator_tuned, X_test, y_test)
## Complete the code to check performance for test data on tuned estimator
bagging_estimator_tuned_model_test_perf
```

```
BaggingClassifier
BaggingClassifier(max_features=0.7, max_samples=0.8, n_estimators=100,
                  random_state=1)
```



	Accuracy	Recall	Precision	F1
0	0.998823	1.0	0.99824	0.999119



	Accuracy	Recall	Precision	F1
0	0.72292	0.894417	0.743043	0.811733

Observation: Model may be slightly overfitted still.

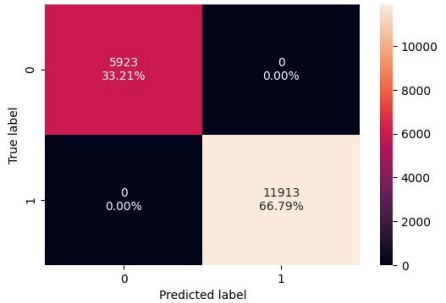
Model Building - Bagging -Random Forest

```
RandomForestClassifier
RandomForestClassifier(class_weight='balanced', random_state=1)
```

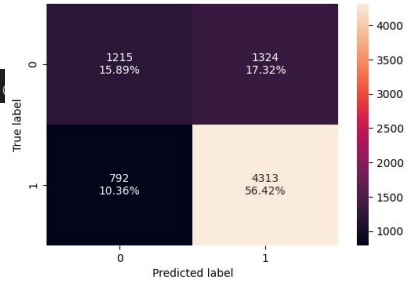
```
# Fitting the model
rf_estimator = RandomForestClassifier(random_state=1, class_weight= 'balanced')
## Complete the code to define random forest with random state = 1 and class_weight = balanced
rf_estimator.fit(X_train, y_train) ## Complete the code to fit random forest on the train data
confusion_matrix_sklearn(rf_estimator, X_train, y_train)
## Complete the code to create confusion matrix for train data

rf_estimator_model_train_perf = model_performance_classification_sklearn
(rf_estimator, X_train, y_train)
## Complete the code to check performance on train data
rf_estimator_model_train_perf
confusion_matrix_sklearn(rf_estimator, X_test, y_test)
## Complete the code to create confusion matrix for test data
rf_estimator_model_test_perf = model_performance_classification_sklearn(rf_estimat
Complete the code to check performance for test data
rf_estimator_model_test_perf
```

	Accuracy	Recall	Precision	F1
0	1.0	1.0	1.0	1.0



	Accuracy	Recall	Precision	F1
0	0.723182	0.844858	0.765123	0.803016



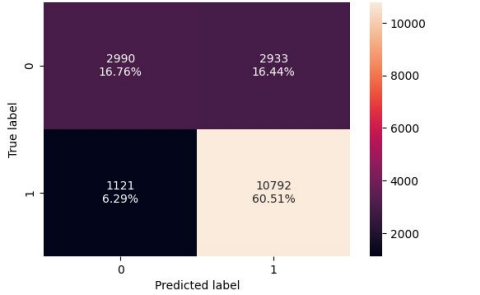
Observation: Model is overfitting.

Model Improvement - Bagging - Random Forest tuned

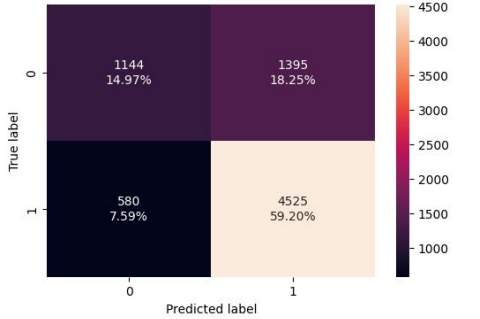
```
grid_obj = GridSearchCV(rf_tuned, parameters, scoring=acc_scorer, cv=5, n_jobs=-1)
## Complete the code to run grid search with cv = 5 and n_jobs = -1
grid_obj = grid_obj.fit(X_train, y_train)
## Complete the code to fit the grid_obj on the train data
confusion_matrix_sklearn(rf_tuned, X_train, y_train)
## Complete the code to create confusion matrix for train data on tuned estimator
rf_tuned_model_train_perf = model_performance_classification_sklearn
(rf_tuned, X_train, y_train) ## Complete the code to check performance
for train data on tuned estimator
Rf_tuned_model_train_perf
confusion_matrix_sklearn(rf_tuned, X_test, y_test)
## Complete the code to create confusion matrix for test data on tuned estimator
rf_tuned_model_test_perf = model_performance_classification_sklearn
(rf_tuned, X_test, y_test)
## Complete the code to check performance for test data on tuned estimator
rf_tuned_model_test_perf
```

```
RandomForestClassifier
RandomForestClassifier(max_depth=10, min_samples_split=7, n_estimators=30,
                       oob_score=True, random_state=1)
```

	Accuracy	Recall	Precision	F1
0	0.772707	0.905901	0.786302	0.841875



	Accuracy	Recall	Precision	F1
0	0.741627	0.886386	0.764358	0.820862



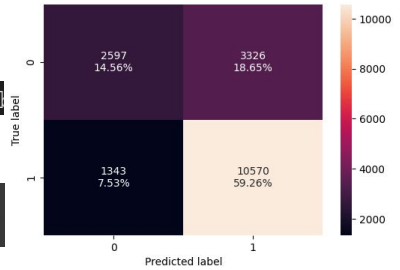
Observation: Model is pretty good.

Model Building - Boosting - AdaBoosting

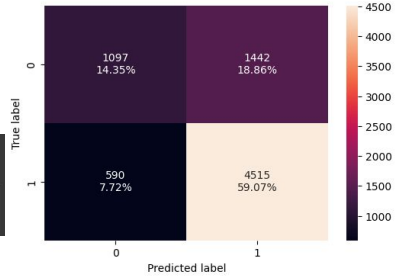
```
ab_classifier = AdaBoostClassifier(random_state=1)
## Complete the code to define AdaBoost Classifier with random state = 1
ab_classifier.fit(X_train, y_train) ## Complete the code to fit AdaBoost Classifier on the train data
confusion_matrix_sklearn(ab_classifier, X_train, y_train)
## Complete the code to create confusion matrix for train data
ab_classifier_model_train_perf = model_performance_classification_sklearn(ab
y_train) ## Complete the code to check performance on train data
ab_classifier_model_train_perf
confusion_matrix_sklearn(ab_classifier, X_test, y_test)
 ## Complete the code to create confusion matrix for test data
ab_classifier_model_test_perf = model_performance_classification_sklearn
(ab_classifier, X_test, y_test)
## Complete the code to check performance for test data
ab_classifier_model_test_perf
```

```
▾ AdaBoostClassifier
AdaBoostClassifier(random_state=1)
```

	Accuracy	Recall	Precision	F1
0	0.738226	0.887266	0.760651	0.819094



	Accuracy	Recall	Precision	F1
0	0.734171	0.884427	0.757932	0.816308



Observation: Model pretty good.

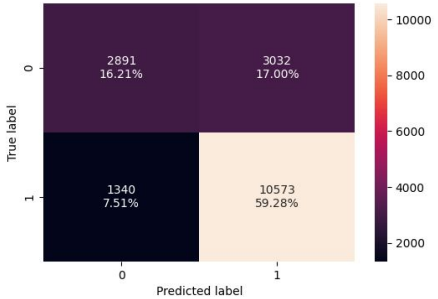
Model Building - Boosting - AdaBoosting Tuned

```
grid_obj = GridSearchCV(abc_tuned, parameters, scoring=acc_scorer, cv=5)
## Complete the code to run grid search with cv = 5
grid_obj = grid_obj.fit(X_train, y_train)
## Complete the code to fit the grid_obj on train data
confusion_matrix_sklearn(abc_tuned, X_train, y_train)
## Complete the code to create confusion matrix for train data on tuned estimator
abc_tuned_model_train_perf = model_performance_classification_sklearn
(abc_tuned, X_train, y_train)
## Complete the code to check performance for train data on tuned estimator
Abc_tuned_model_train_perf
confusion_matrix_sklearn(abc_tuned, X_test, y_test)
## Complete the code to create confusion matrix for test data on tuned estimator
abc_tuned_model_test_perf = model_performance_classification_sklearn
(abc_tuned, X_test, y_test)
## Complete the code to check performance for test data on tuned estimator
abc_tuned_model_test_perf
```

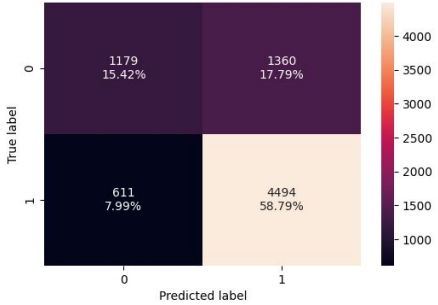
Observation: Model is pretty good.

AdaBoostClassifier
base_estimator: DecisionTreeClassifier
DecisionTreeClassifier

	Accuracy	Recall	Precision	F1
0	0.754878	0.887518	0.777141	0.82867

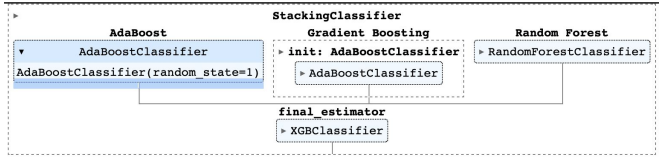


	Accuracy	Recall	Precision	F1
0	0.742151	0.880313	0.76768	0.820148



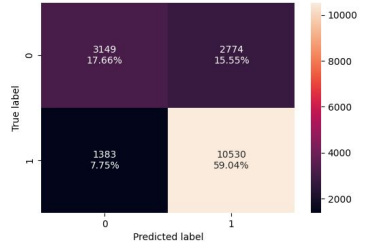
Model Improvement - Stacking Classifier

```
stacking_classifier = StackingClassifier(estimators=estimators, final_estimator=final_estimator)
## Complete the code to define Stacking Classifier
stacking_classifier.fit(X_train, y_train)
## Complete the code to fit Stacking Classifier on the train data
```



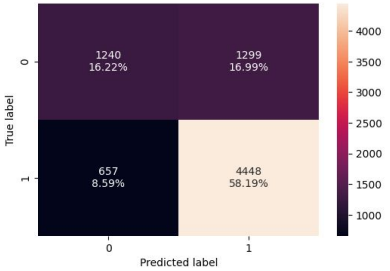

```
confusion_matrix_sklearn(stacking_classifier, X_train, y_train) ## Complete the code to create confusion matrix for train data
stacking_classifier_model_train_perf = model_performance_classification_sklearn(stacking_classifier, X_train, y_train)
## Complete the code to check performance on train data
stacking_classifier_model_train_perf
```

	Accuracy	Recall	Precision	F1
0	0.766932	0.883908	0.791491	0.835151




```
confusion_matrix_sklearn(stacking_classifier, X_test, y_test)
## Complete the code to create confusion matrix for test data
stacking_classifier_model_test_perf = model_performance_classification_sklearn(stacking_classifier, X_test, y_test)
## Complete the code to check performance for test data
stacking_classifier_model_test_perf
```

	Accuracy	Recall	Precision	F1
0	0.744113	0.871303	0.773969	0.819757



Observation: Model is pretty good.

Slide Header

- Please add any other pointers (if needed)

Great Learning

Happy Learning !